

5 AI Prompts That Write Playwright Tests for You

Free Resource by Authority OS

Copy any prompt below, paste it into Claude, ChatGPT, or your AI of choice, fill in the bracketed sections, and get production-ready Playwright code in seconds.

Prompt 1 — Generate a Complete E2E Test from a User Story

Use this when: you have a feature description and need a full test file immediately.

You are a Senior QA Automation Engineer using Playwright and TypeScript.

I need an E2E test for this scenario:

[DESCRIBE THE USER JOURNEY IN PLAIN ENGLISH]

Requirements:

- Use Page Object Model (separate locators from test logic)
- Follow AAA pattern: Arrange, Act, Assert
- Use web-first assertions (`expect(locator).toBeVisible()`)
- No hardcoded credentials – use `process.env`
- Add `@smoke` tag if this is a critical business path

Pages involved: [LIST THE PAGES, e.g. Login, Cart, Checkout]

Return: the complete `.spec.ts` file and any Page Object updates needed.

Example input: "User adds two items to cart, fills in checkout info, and sees the order confirmation page"

What you get: A complete spec file with imports, describe block, test steps, and assertions — ready to run.

Prompt 2 — Fix a Flaky Test in 30 Seconds

Use this when: a test fails intermittently in CI and you can not reproduce it locally.

You are a Playwright expert diagnosing test flakiness.

Here is my failing test:

[PASTE YOUR TEST CODE]

Here is the error from CI:

[PASTE THE ERROR MESSAGE]

Browser: [CHROME / FIREFOX / WEBKIT]

Diagnose the root cause (race condition, timing issue, strict mode violation, etc.) and return the fixed test with a comment explaining what was wrong.

What you get: Fixed code + explanation. Most flaky tests are caused by 3 things: missing `await`, asserting before the UI updates, or selectors that match multiple elements. The AI spots all three instantly.

Prompt 3 — Create a Page Object from an HTML Snippet

Use this when: you have a new page to test and need the Page Object class fast.

You are a QA Architect building a Playwright Page Object Model class.

Here is the HTML (or description) of the page:
[PASTE HTML SNIPPET OR DESCRIBE THE UI]

Page name: [e.g. CheckoutPage]

Requirements:

- Class extends BasePage
- Locator priority: getByRole > getByTestId > getByText > getByLabel
- No CSS selectors or XPath
- Every method wrapped in test.step() for Allure logging
- Include: constructor, goto(), and one method per user action

Return the complete TypeScript class.

What you get: A production-ready Page Object with correct locators, typed methods, and Allure step logging.

Prompt 4 — Turn a Bug Report into a Test

Use this when: QA or a user found a bug and you want to prevent regression.

You are a QA Engineer writing a regression test to prevent a bug from recurring.

Bug report:

Title: [BUG TITLE]

Steps to reproduce: [NUMBERED STEPS]

Expected: [WHAT SHOULD HAPPEN]

Actual: [WHAT HAPPENS INSTEAD]

Write a Playwright test that:

1. Reproduces the exact bug scenario
2. Asserts the FIXED behaviour (so the test passes when the bug is fixed)
3. Has a comment referencing this bug: // Regression: [BUG ID]
4. Uses Page Objects – no raw locators in the test file

Return the complete test file.

What you get: A regression test that permanently protects against this bug reappearing.

Prompt 5 — Generate a Smoke Suite for Any Web App

Use this when: you are starting a new project and need a smoke suite from scratch.

You are a QA Architect designing a smoke test suite.

Application: [DESCRIBE THE APP IN 2-3 SENTENCES]

Tech stack: [e.g. React, REST API, Postgres]

Critical user journeys (must work for the business to function):

1. [JOURNEY 1]
2. [JOURNEY 2]
3. [JOURNEY 3]

Design a smoke suite that:

- Runs in under 5 minutes
- Covers every revenue-critical path
- Uses @smoke tag on all tests
- Follows AAA pattern
- Uses Page Object Model

Return: the smoke suite plan (table) + empty test shells ready to fill in.

What you get: A prioritised smoke plan + empty spec files with the right structure — you just fill in the selectors.

Want the Full Toolkit?

These 5 prompts are extracted from the **Authority OS AI QA Toolkit** — a complete framework with:

- 15+ production-ready AI prompts
- Playwright + TypeScript framework (POM, cross-browser, CI/CD)
- Notion dashboard that auto-updates after every test run
- Live Allure report published on every push
- ROI calculator that shows your team the business value of automation

Authority OS — [Get the full toolkit]

Created by Authority OS Dev / authority.os.dev@gmail.com